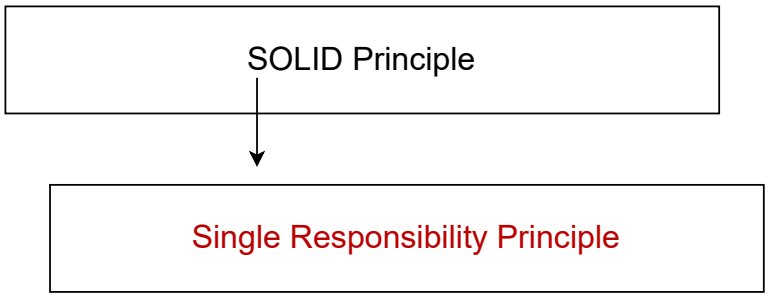
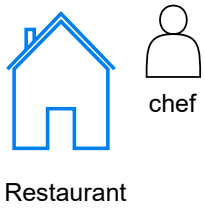
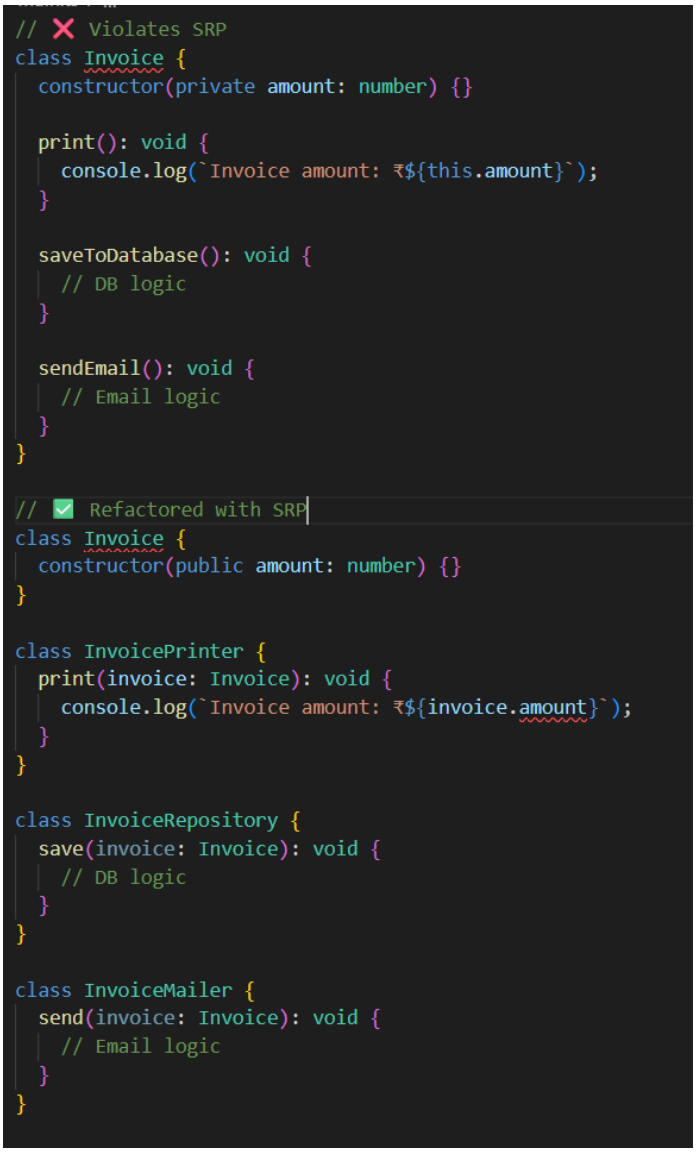
SOLID Principle

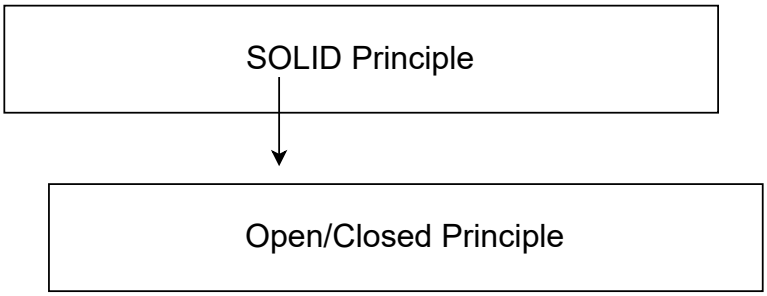


" A class should have only one reason to change "

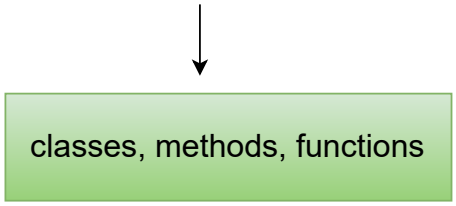


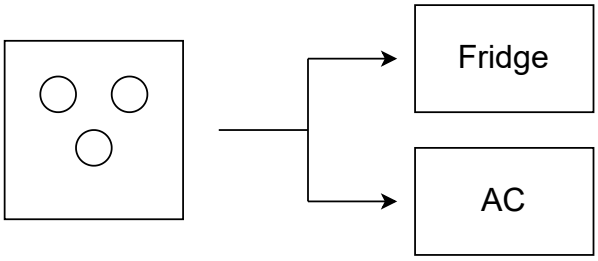


```ts
// ❌ Violates SRP
class Invoice {
  constructor(private amount: number) {}

  print(): void {
    console.log(`Invoice amount: ₹${this.amount}`);
  }

  saveToDatabase(): void {
    // DB logic
  }

  sendEmail(): void {
    // Email logic
  }
}

// ✅ Refactored with SRP
class Invoice {
  constructor(public amount: number) {}
}

class InvoicePrinter {
  print(invoice: Invoice): void {
    console.log(`Invoice amount: ₹${invoice.amount}`);
  }
}

class InvoiceRepository {
  save(invoice: Invoice): void {
    // DB logic
  }
}

class InvoiceMailer {
  send(invoice: Invoice): void {
    // Email logic
  }
}
```

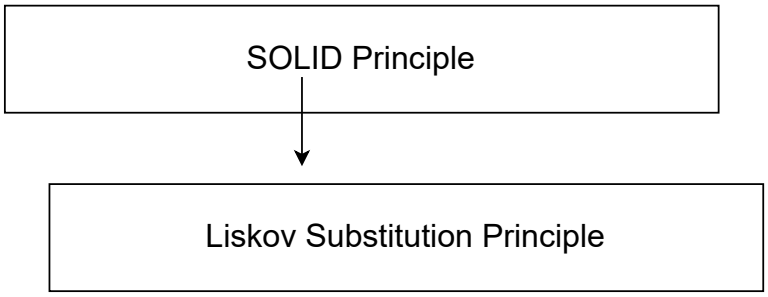


“Software entities should be open for extension but closed for modification.”



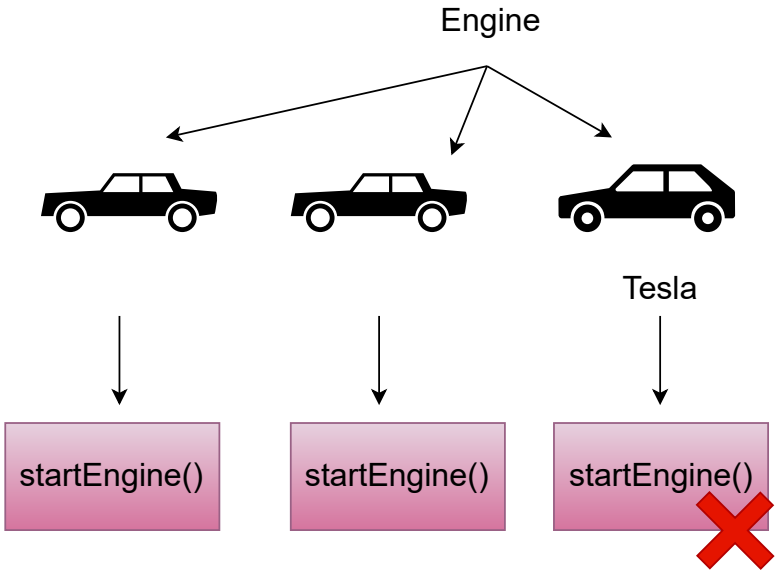


```ts
// ❌ Violates OCP
class Discount {
  getDiscount(type: string): number {
    if (type === 'student') return 10;
    if (type === 'senior') return 15;
    return 0;
  }
}

// ✅ Refactored with OCP
interface DiscountStrategy {
  getDiscount(): number;
}

class StudentDiscount implements DiscountStrategy {
  getDiscount(): number {
    return 10;
  }
}

class SeniorDiscount implements DiscountStrategy {
  getDiscount(): number {
    return 15;
  }
}

class DiscountCalculator {
  constructor(private strategy: DiscountStrategy) {}

  calculate(): number {
    return this.strategy.getDiscount();
  }
}

const discount = new DiscountCalculator(new StudentDiscount())
discount.calculate()
```



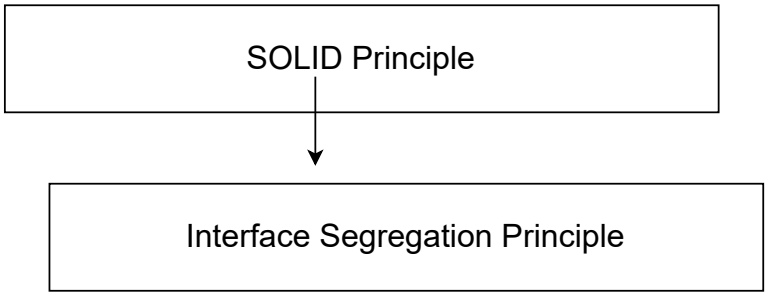
“Subtypes must be substitutable for their base types.”

“If you replace a parent class with a child class, everything should still work.”

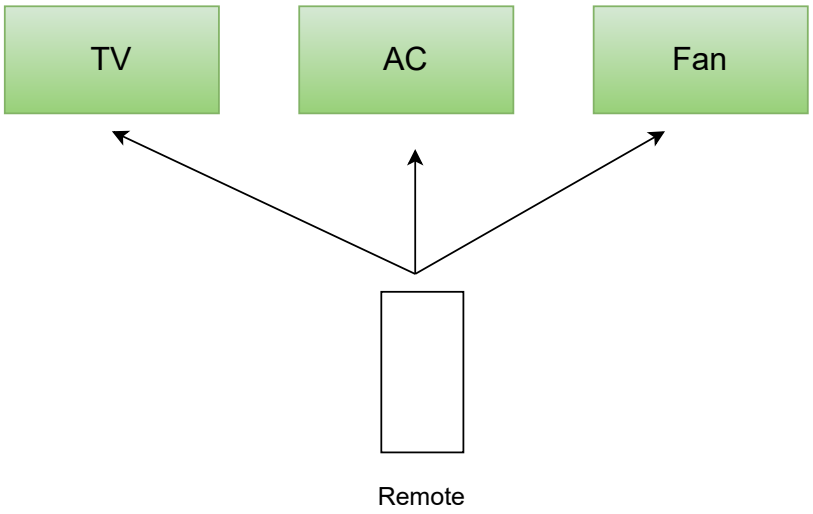


```ts
// ❌ Violates
class Notifier {
  send(message: string): void {
    console.log("Sending message:", message);
  }
}

class SMSNotifier extends Notifier {
  send(message: string): void {
    console.log("Sending SMS:", message);
  }
}

class PushNotifier extends Notifier {
  send(message: string): void {
    throw new Error("Device token missing");
  }
}

// Somewhere in the app
function notifyUser(notifier: Notifier) {
  notifier.send("Hello!");
}
```

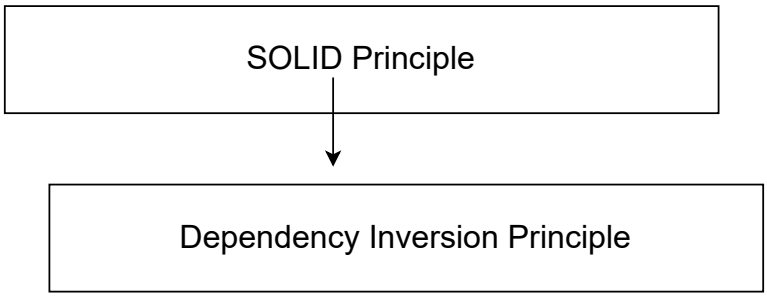
```ts
// ✅ Refactored
interface Notifier {
  send(message: string): void;
}

class SMSNotifier1 implements Notifier {
  send(message: string): void {
    console.log("Sending SMS:", message);
  }
}

class PushNotifier1 implements Notifier {
  constructor(private deviceToken?: string) {}

  send(message: string): void {
    if (!this.deviceToken) {
      console.log("Skipping push: no device token");
      return;
    }
    console.log("Sending push:", message);
  }
}

function notifyUser1(notifier: Notifier) {
  notifier.send("Hello!");
}
```



"Clients should not be forced to depend on interfaces they do not use.”

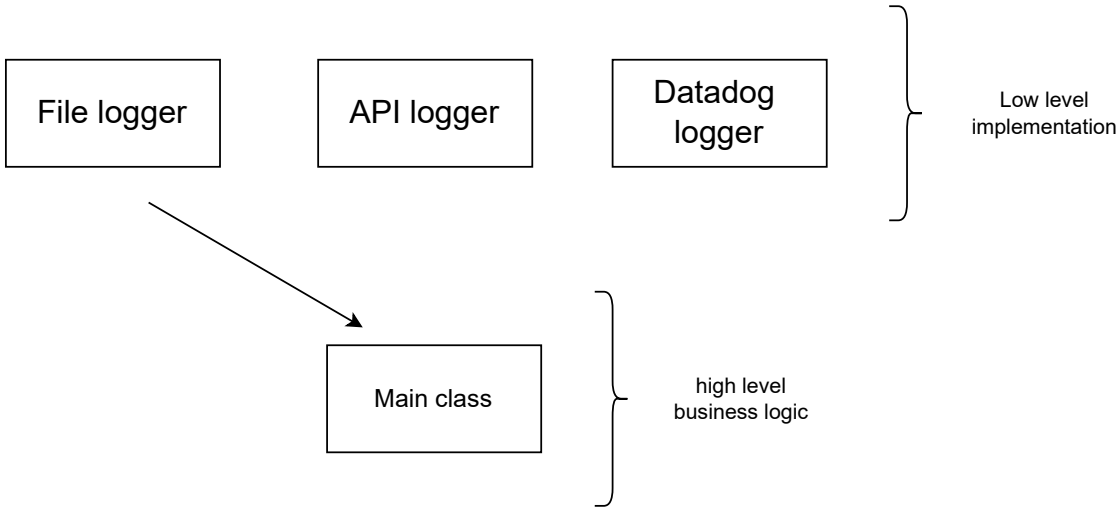


```ts
// ❌ Violates ISP
interface Machine {
  print(): void;
  scan(): void;
  fax(): void;
}

class OldPrinter implements Machine {
  print(): void {}
  scan(): void {
    throw new Error("Not supported");
  }
  fax(): void {
    throw new Error("Not supported");
  }
}

// ✅ Refactored with ISP
interface Printer {
  print(): void;
}

interface Scanner {
  scan(): void;
}

class BasicPrinter implements Printer {
  print(): void {}
}
```



"Depend on abstractions, not concretions."

High-level modules should not depend on low-level modules. Both should depend on **abstractions**.



```ts
// ❌ Violates
class FileLogger {
  log(message: string): void {
    console.log("Writing to file:", message);
  }
}

class UserService {
  private logger = new FileLogger(); // tightly coupled

  createUser(): void {
    this.logger.log("User created");
  }
}
```

```ts
// ✅ Refactored
// Abstraction
interface Logger {
  log(message: string): void;
}

// Low-level implementations
class FileLogger implements Logger {
  log(message: string): void {
    console.log("Writing to file:", message);
  }
}

class DatadogLogger implements Logger {
  log(message: string): void {
    console.log("Sending to Datadog:", message);
  }
}

// High-level module
class UserService {
  constructor(private logger: Logger) {}

  createUser(): void {
    this.logger.log("User created");
  }
}
```